

Proyecto 2 Principios de Sistemas Operativos

Ángel Eduardo Jiménez Valverde*

*Ingeniería en Computación, ITCR, Cartago, Costa Rica

Email: 436942@estudiantec.cr

Carnet: 2021436942

I. RESPUESTAS DEL QUIZ PASADO

- 1) ¿Qué es una capa de convolución y una capa de pooling?
- 2) Describa parte por parte un módulo de Inception:
- 3) Describa como puede usarse un autoencoder para detección de anomalías:
- 4) Describa la composición de una arquitectura U-Net:

II. INFORMACIÓN IMPORTANTE ACERCA DEL CURSO

- En esta semana el jueves 28 de Marzo se nos asignará la tarea final del curso. Ésta tarea consistirá en el manejo de agentes de LLM, utilizando como recursos los apuntes que se han hecho a través del semestre en éste curso.
- En la semana 15 se nos asignará el proyecto de Inteligencia Artificial.
- El examen del curso de Inteligencia Artificial se hará en semana 18, a las 5:00 p.m.

III. REPASO DE LA CLASE PASADA

En la clase pasada aprendimos acerca de los Lenguajes de Alto Lenguaje (LLM). Aprendimos que el texto u información inicialmente no es usado directamente, sino que se requiere convertir en **tokens**.

Estos **tokens** pueden estar formados ya sea por palabras, semipalabras o incluso letras, y dependiendo del método que se utilice, es más práctico (siendo el más común el método de semipalabras).

TABLE I
TIPOS DE FORMAS DE TOKENIZAR Y SUS VENTAJAS

Tipo	Ejemplo	Ventaja Principal
Palabra	"Los modelos"	Simplicidad
Carácter	"L", "o", "s"	Sin OOV*
Subpalabra (BPE, WordPiece)	"aprend", "iendo"	Equilibrio vocabulario/-contexto
Byte-level	bytes UTF-8	Soporta cualquier símbolo
Espacio en Blanco	"Hola", "mundo"	Rápido y simple

Una vez tokenizadas, cada uno de éstos tokens tendrán una representación o valor numérico, dichos valores de identificación los conoceremos como **embeddings**, y nos permitirán

3D Semantic Feature Space

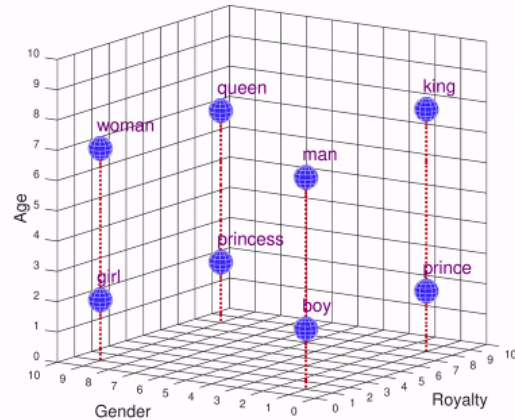


Fig. 1. Gráfico Semántico de 3 dimensiones con 3 características (género, edad, realeza)

representar de forma matemática su nivel de similitud y contexto luego, mediante el gráfico de n dimensiones.

Una vez tenemos todos nuestros vectores, ocuparemos calcular los niveles de similitud de cada punto, para poder calcular un nivel de contexto, por poner un ejemplo: **un rey** y un **príncipe** ambos tienen el mismo *género* y *realeza*, pero difieren en *edad*.

El algoritmo que se usa para calcular éstos niveles de similitudes es el siguiente:

$$s(a, d) = \frac{a \cdot b}{||a|| \cdot ||b||}$$

Ésta fórmula nos permite sacar un ángulo, que luego al calcularse en el coseno podemos decidir si:

- Entre más cerca a 1, más similar es
- Entre más cerca a 0, menos similitudes puede llegar a ser
- Entre más cerca a -1, el valor es completamente opuesto.

Una vez conociendo esto, tenemos que reconocer las principales debilidades que tienen los LLM: **Conocimiento estático** y **Alto nivel de alucinaciones**. Pero, utilizando el RAG(*Retrieval-Augmented Generation*) somos capaces de solucionar éstos posibles errores. Éste se forma por los siguientes pasos:

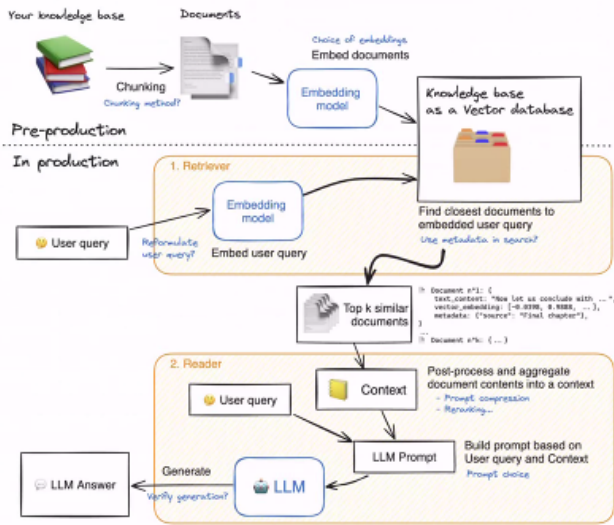


Fig. 2. Representación del cómo se vería el actuar del RAG

A. Ingesta y Chunking

- Preparar los documentos donde se va a recuperar la información.
- Dividimos el texto en **chunks** (fragmentos entre 200-500 tokens o más).
- Transformamos los chunks con un modelo de embeddings en vectores, para capturar el significado semántico de cada uno.
- Al generar los vectores, comparamos la pregunta generada, la convertimos en un **embedding**, y usamos el algoritmo de similitud con dicha pregunta como el prompt, y buscamos la respuesta cuyo nivel de similitud sea más cercano a 1, una vez pase ésto, se retorna ese dato con la metadata exacta de su ubicación en el vector, y se le da al sistema para que la retorne al usuario.

IV. APUNTES DE LA CLASE DE HOY

A. Agentes

Supongamos que se nos brinda un escenario: *Eres un programador al que le dan 10 días de vacaciones para fecha de navidad (o hasta que uno de los servidores de la empresa se caigan), y ocupas información para planear dichas vacaciones, usando inteligencia artificial.*

Para ésto, vas a utilizar distintos niveles de LLM, hasta buscar uno que sea el más práctico para tu situación.

En éste caso, la mejor opción sería el agente, no solamente porque ahora es un sistema conversacional autónomo, que será capaz de analizar la situación laboral, la temporada de vacaciones, información acerca de los gustos del usuario (y más información que se le sea dada).

Sino que incluye otras ventajas, entre esas:

TABLE II
NIVELES EVOLUTIVOS DE UN LLM

Nivel Evolutivo	Ventaja Principal
LLM Tradicional	Es útil para obtener información general, pero no para personalizar ni actuar
Usando RAG	Ahora el sistema puede acceder a mi información laboral o personal, pero sigue sin poder tomar decisiones
Agente	Es la evolución final, el sistema no solo responde, sino que es capaz de razonar y actuar, pasando a ser un asistente autónomo

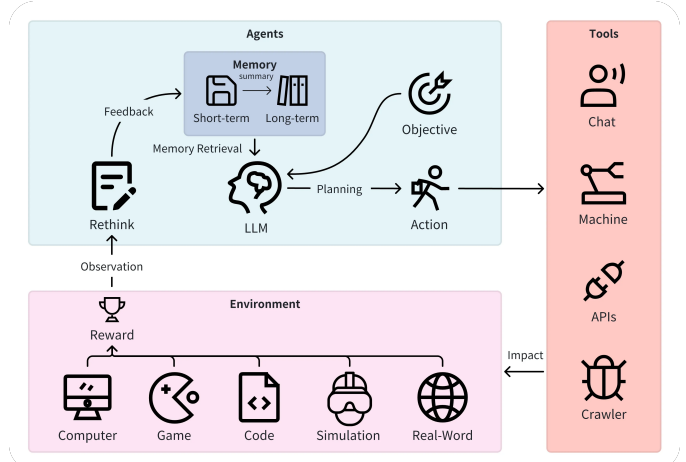


Fig. 3. Representación visual del cómo funciona un Agente

- **Memoria:** Recuerda mis preferencias (ejemplo: Destinos anteriores).
- **Herramientas:** Consulta API's de vuelos, clima, calendario, hospedaje, estado de los servidores.
- **Planificación:** Organiza un itinerario según mis fechas y presupuesto.

B. Perfil del agente

Inicialmente requeriremos definir la personalidad del agente, su comportamiento y nivel de rol.

Durante la clase pudimos observar que dependiendo del cómo implementamos o modificamos el perfil, éste puede ser más práctico para algunas acciones, por ejemplo tenemos algunos de las mejores herramientas de IA para respectivas acciones. **Checar Tabla III**

C. Tipos de modelos utilizados en agentes

Existen 2 enfoques principales para utilizar modelos de lenguaje en agentes:

- Modelos propietarios (closed source)
- Modelos abiertos (open source / open-weight)

TABLE III
DISTINTOS MODELOS LLM Y SUS RATINGS DE ACUERDO A SU
RENDIMIENTO EN ACTIVIDADES

AI Tool	Tool Calling Rating	Writing Rating
Deepseek v4-Pro	34.2	X
GPT-5.4	35.3	35.5
Claude Opus 4.6	34.5	44.6
Claude Mythos Preview 4.6	39.8	X
Gemini 3.5 Flash	42.7	X
LongCat-Flash-Thinking	X	38.0
Qween2.5 72B Instruct	X	31.0
Llama 3.1 405B Instruct	40.5	X
Hermes 3-70DB	X	34.5

Ambos enfoques pueden ser utilizados para construir agentes modernos. La decisión dependerá de los siguientes atributos:

- Requerimientos técnicos
- Costos
- Privacidad
- Infraestructura disponible

1) *Modelos Proprietarios (Closed Source):*

- **Ventajas:**
 - Fácil integración
 - Infraestructura administrada
 - Buen rendimiento general
 - Soporte avanzado para herramientas
- **Desventajas:**
 - Dependencia del proveedor
 - Costos por uso
 - Menor control sobre el modelo

2) *Modelos abiertos (open source / open-weight):*

- **Ventajas:**
 - Mayor control
 - Mejor privacidad
 - Personalización
 - Posibilidad de ejecución local
- **Desventajas:**
 - Mayor complejidad operativa
 - Requieren infraestructura
 - Configuración y monitoreo propios

Uno de los retos más importantes al construir un agente es decidir. ¿Qué modelo de lenguaje deberíamos utilizar?

La verdad es que no existe un único modelo ideal para todas las tareas, el mejor modelo varía dependiendo de las tareas, y principalmente de los atributos que se tengan.

Al final, seleccionar el LLM dependerá del problema que queremos solucionar, la infraestructura y las restricciones que tendrá el sistema.

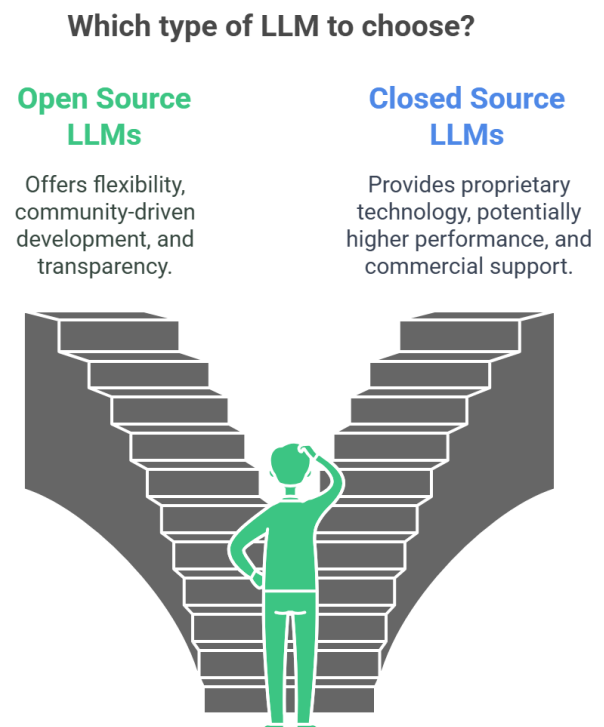


Fig. 4. Aspectos para elegir el tipo de LLM para un agente